PYTHON · FROM ZERO · LEARNING LOG

Python Cheat Sheet

Everything I've learned so far, in one place: each topic with examples, the exact traps I fell into, and how to avoid them. **Revise one section a day.**

7
lessons done

11
LeetCode solved

13
HackerRank solved

27 Oct 2026
last updated

Contents

0 · How to revise & solve
daily routine, problem-solving method, checklist

1 · print, variables, f-strings
sep, end, :.2f

2 · Types & input
int, float, str, None, input() is text

3 · Operators & conditions
// % **, = vs ==, and/or, in, if/elif

4 · Loops ⭐
range, while, break/continue/pass, items vs positions

5 · Strings
index, slice, methods, compare, split/join

6 · Lists ⭐
append, sort vs sorted, in place, nested

7 · Functions
return vs print, None, defaults, scope

8 · Dictionaries
get, items, counting pattern

9 · Parsing input ⭐
split, map, name + numbers, many lines

10 · Pattern library
accumulator, filter, best so far, two pointers…

11 · Error decoder
what each error message really means

12 · LeetCode & HackerRank format
class Solution, in place, timer noise

13 · Git daily commands
stage, commit, push, partial staging

14 · Problems solved
links to every problem

0 · How to revise & solve

↑ contents

🗓️ **Daily routine (about 10 min)**

- Read **one section** of this sheet, top to bottom
- Cover the ✅ boxes and predict the answer to each ❌ box
- Then: 1 LeetCode + 1 commit (the streak minimum)

🧯 **When panic hits**

- Stop typing. Solve the sample **by hand** first
- Comment out broken code with `#`, don't delete it
- Fix the **first** red squiggle only, then re-run

The 6-step method

#	Step	What it looks like
1	Solve the sample by hand	paper, no code
2	Write the steps as `# STEP` comments	a big problem becomes 4–5 small ones
3	Write each step's data shape	`# students = [["Harry", 37.21], ...]`
4	Code ONE step, `print()`, run	check the shape matches, then the next step
5	Translate with the phrasebook (below)	English → Python
6	Remove debug prints, test tricky inputs, submit	duplicates, uppercase, empty, boundaries

Phrasebook: English → Python

I want to...	Python
start an empty list / dict / text / total	`x = []` · `d = {}` · `s = ""` · `total = 0`
add to a list	`x.append(item)`
go through every item	`for item in x:`
go through every position	`for i in range(len(x)):` → `x[i]`
keep only some items	`if condition:` then `append`
first / last item	`x[0]` / `x[-1]`
stop at the first match	`break` (or `return` in a function)
count things	`counts[k] = counts.get(k, 0) + 1`
swap two items	`x[a], x[b] = x[b], x[a]`

✅ Checklist before Run / Submit

- Is the loop variable an **item** or a **position**? Did I name it after what it is?
- Does my `range` give exactly the right first and last values?
- Am I **replacing** (`=`) or **adding on** (`+=`)?
- Text or number? Do I need `int()` / `float()` / `str()` ?
- Could uppercase break it? Use `.lower()`
- Want the first match? Did I `break` ?
- Does every `while` change something so it can stop?
- Does my function **return** (not print)?
- Did I test a tricky input (duplicates, boundaries like exactly 90, empty)?

1 · print, variables, f-strings

↑ contents

```
name = "Gowri"            # a variable is a labelled box holding a value
age = 31
print("My name is", name)  # My name is Gowri    (commas → a SPACE)
print(f"Next year: {age + 1}")   # Next year: 32  (f-string: {} is replaced)
print("A", "B", "C", sep="-")    # A-B-C          (sep = what goes between)
print("Hi", end="")              # no newline, the next print continues here
print("!")                       # Hi!
avg = 26.5
print(f"{avg:.2f}")              # 26.50          (2 decimal places)
```

❌ What I did

```
print("age", age "next", age+1)   # SyntaxError
print(f"{robot} is{robot_age}")   # Optimus is2
```

✅ Right

```
print("age", age, "next", age + 1)
print(f"{robot} is {robot_age}")  # spaces print as typed
```

🔑 **Remember**

- The commas in `print(a, b)` are **code**: Python prints a **space**. Only text inside quotes (or `sep=`) appears on screen.
- `{x}` means "insert value" **only inside f-strings**.

2 · Types & input

↑ contents

Type	Example	Convert with
int (whole number)	`31`	`int("31")` → `31`
float (decimal)	`37.21`	`float("37.21")`
str (text)	`"Gowri"`	`str(5)` → `"5"`
bool	`True` / `False`	comparisons give these
None ("nothing")	`None`	what a function without `return` gives (other languages call it `null`)

```
age = int(input("Age? "))   # input() ALWAYS returns TEXT, so convert it
print(type(age))              # <class 'int'>   (type() shows what you're holding)
```

❌ Trap

```
age = input("Age? ")
print(age + 1)        # TypeError: text + number
"12" + 3              # TypeError
```

✅ Right

```
age = int(input("Age? "))
print(age + 1)
"12" + str(3)         # "123"
```

3 · Operators & conditions

↑ contents

Op	Example	Result	Meaning
`/`	`7 / 2`	3.5	float division, always a decimal (`8 / 2` → 4.0)
`//`	`7 // 2`	3	integer division, drops the decimal
`%`	`7 % 2`	1	remainder. `n % 2 == 0` → even
`**`	`3 ** 2`	9	power
`+= -= *=`	`x += 1`		shortcut for `x = x + 1`

= vs ==

```
x = 5          # STORE 5 in x
x == 5         # ASK "is x 5?" → True
```

Combining conditions

```
if 2 ≤ n ≤ 5:           # range check in one go
if x > 0 and x < 10:    # both true
if ch in "aeiou":       # one of these?
```

Flat if / elif / else

```
if n % 2 == 1:
    print("Weird")
elif 2 ≤ n ≤ 5:         # only checked if everything
above was False
    print("Not Weird")
elif 6 ≤ n ≤ 20:
    print("Weird")
else:
    print("Not Weird")
```

❌ Traps I fell into

```
if ch == 'a' | 'e':          # | is NOT "or"
if ch == 'a' or 'e':         # ALWAYS True!
return grade == "A"          # returns False, == compares
if score > 90:               # 90 itself excluded
```

✅ Right

```
if ch in "aeiou":
if ch == 'a' or ch == 'e':
return "A"
if score ≥ 90:               # "90+" includes 90
```

4 · Loops ⭐

↑ contents

🔴 **My #1 mistake (8 times!): items vs positions**

Loop	Gives you	Use it as
`for num in nums:`	**items**: 1, 2, 1	`num` directly
`for i in range(len(nums)):`	**positions**: 0, 1, 2	`nums[i]`

Before writing `x[something]`, ask: **"is `something` a position or an item?"** If it came from `for something in x`, it's an item. **Name loop variables after what they are** (`num`, `name` = item · `i` = position).

❌ What I did (LeetCode 1929)

```
nums = [1, 2, 1]
for i in nums:
    ans.append(nums[i])   # nums[1], nums[2], nums[1]
# → [2, 1, 2]  wrong!
```

✅ Right

```
for num in nums:
    ans.append(num)
# → [1, 2, 1]
```

range()

Code	Gives
`range(5)`	0, 1, 2, 3, 4 (stops **before** 5)
`range(1, 6)`	1, 2, 3, 4, 5
`range(2, 21, 2)`	2, 4, … 20 (step)
`range(len(x) - 1, -1, -1)`	positions backwards: 4, 3, 2, 1, 0
`range(nums)` ❌	TypeError: range needs a **number** → `range(len(nums))`

while

```
count = 3
while count > 0:
    print(count)
    count -= 1      # MUST change, or it runs forever
print("Liftoff!")
```

TLE "Time Limit Exceeded" usually means an endless `while`.

break · continue · pass

```
for n in [1, 2, 3, 4]:
    if n == 2:
        continue    # skip this round
    if n == 4:
        break       # stop the WHOLE loop
    print(n)        # 1, 3
# pass = do nothing (placeholder)
```

❌ Other loop traps

```
for name in ["A", "B", "C"]:
    pass
print(name)      # "C": keeps the LAST value!

for letter in s:           # loop never needed
    while first < last: …  # reverses len(s) times
```

✅ Remember

- Use the variable that holds **what the question asks for**, not a leftover loop variable.
- Off-by-one: `range(0, n + 1)` gives n+1 items. `range(n)` gives exactly n.
- `for _ in range(n):` → "repeat n times, I don't need the counter".
- Loop variable never used? Ask "do I need this loop at all?"

5 · Strings

↑ contents

```
robot = "Optimus"
#  O  p  t  i  m  u  s
#  0  1  2  3  4  5  6
# -7 -6 -5 -4 -3 -2 -1
len(robot)      # 7
robot[0]        # "O"        robot[-1]   # "s"
robot[0:3]      # "Opt"      (stops BEFORE 3, like range)
robot[3:]       # "imus"     robot[::-1] # "sumitpO"
(reversed)
```

Method	Example → result
`.upper()` / `.lower()`	`"Hi".upper()` → `"HI"`
`.strip()`	`" hi ".strip()` → `"hi"`
`.replace(old, new)`	`"1.2.3".replace(".", "[.]")` → `"1[.]2[.]3"`
`.count(x)`	`"banana".count("a")` → `3`
`.split()`	`"I love code".split()` → `['I', 'love', 'code']` (a **list of strings**)
`"sep".join(list)`	`" ".join(['I', 'love'])` → `"I love"`
`x in text`	`"ank" in "Sankar"` → `True`

🔑 Strings can't change

Methods return a **new** string. Save it: `name = name.strip()`. Building text with `s = s + ch` in a loop makes a new string every round, so for huge texts collect a list and `"".join()` once.

🔑 Comparing is case-sensitive

```
"Racecar" == "racecaR"   # False
w = word.lower()
w == w[::-1]             # palindrome ✅
```

❌ What I did

```
for ch in word:
    word[ch]          # ch is a LETTER → TypeError
i.index[i]            # methods use ( ) not [ ]
```

✅ Right

```
for ch in word:
    print(ch)         # use the letter directly
"Robot"[::-1].lower() # chaining: left → right →
"tobor"
```

6 · Lists ⭐

↑ contents

Code	Does
`x.append(v)`	add to the end (the ONLY way to grow an empty list)
`x.insert(i, v)`	add at position `i`
`x.remove(v)`	remove the first item equal to `v` (by **value**)
`x.pop()` / `x.pop(0)` / `x.pop(i)`	remove last / first / at **position** i, and **give it back**: `picked = x.pop(0)` . ❌ `x.pop(x[0])` uses a value as a position
`x[i] = v`	replace an EXISTING slot
`x.sort()` / `x.reverse()`	change the list itself, return `None`
`sorted(x)`	NEW sorted list, original unchanged
`[1, 2] + [3]` · `[1, 2] * 2`	`[1, 2, 3]` · `[1, 2, 1, 2]`
`len(x)` `sum(x)` `max(x)` `min(x)`	functions go AROUND the list

❌ What I did

```
ans = []
ans[0] = 5            # IndexError: no slot 0
x = [3, 1, 2].sort()  # x is None!
scores.max()          # AttributeError
s = s[::-1]           # LeetCode's list unchanged
list_ns.sorted()      # sorted goes AROUND
```

✅ Right

```
ans = []
ans.append(5)
x = sorted([3, 1, 2]) # [1, 2, 3]
max(scores)
s[:] = s[::-1]        # or s.reverse()
sorted(list_ns)
```

📚 **In place vs new (the bookshelf)**

`.sort()`, `.append()`, `.reverse()`, `s[i] = …` **rearrange the same shelf**. `sorted()`, `s[::-1]`, `+` make a **new shelf**. `s = …` only moves the *label* to a new shelf.

Nested lists

```
students = [["Harry", 37.21], ["Tina", 37.2]]
students[0]        # ["Harry", 37.21]
students[0][1]     # 37.21
for name, score in students:
    print(name, score)
```

⚠️ **Never call a variable `list`, `str`, `dict`, `int`**

`len(list)` → "object of type 'type' has no len()". Those names belong to Python. Use the parameter name (`nums`).

7 · Functions

↑ contents

```python
def power(base, exp=2):    # parameters, with a default value
    return base ** exp     # return HANDS BACK the value

result = power(3)          # 9
print(power(2, 5))         # 32
```

❌ print instead of return

```python
def average(nums):
    print(sum(nums) / len(nums))

avg = average([1, 2, 3])   # shows 2.0
print(avg)                 # None 😱
```

✅ Calculate and return, print outside

```python
def average(nums):
    if not nums:           # GUARD CLAUSE
        return 0
    return sum(nums) / len(nums)

print(average([1, 2, 3]))  # 2.0
```

	`break`	`return`
Exits	the nearest **loop** (not the `if`)	the **whole function**, loops included
Code after it	the rest of the function runs	nothing runs (a `break` after `return` is dead code)

🔑 **Rules**

- No `return` reached → the function gives `None` ("falls off the end")
- Variables made inside stay inside → use `return`
- `return` only works inside `def`
- Early returns replace `elif` chains: each `if` can assume the earlier ones failed

⚠️ **Side effects**

`scores.sort()` inside a function also sorts the **caller's** list. Use `scores = sorted(scores)` unless changing it is the job.

```
def grade(score):           # early returns, no elif needed
    if score ≥ 90:
        return "A"
    if score ≥ 75:
        return "B"
    if score ≥ 50:
        return "C"
    return "F"
```

8 · Dictionaries

↑ contents

```
robot = {"name": "Optimus", "year": 2022}   # key: value pairs
robot["name"]              # "Optimus"   (look up by KEY)
robot["price"] = 20000     # add (or change, same syntax)
del robot["year"]          # delete
"name" in robot            # True        (checks KEYS)
len(robot)                 # number of pairs
for key, value in robot.items():
    print(f"{key}: {value}")
```

❌ Missing key crashes

```
robot["height"]    # KeyError: 'height'
for k in robot:    # gives KEYS only
    print(k)
```

✅ Safe

```
robot.get("height")          # None
robot.get("height", "n/a")   # your default
for k, v in robot.items():   # both
```

⭐ The COUNTING pattern = a tally sheet 🖍️

```
counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1   # "current number (or 0) + 1"
print(counts)                            # {'b': 1, 'a': 3, 'n': 2}
```

Round	ch	On sheet?	counts after
1	b	no → 0 + 1	`{'b': 1}`
2	a	no → 0 + 1	`{'b': 1, 'a': 1}`
4	a	**yes** → 1 + 1	`{'b': 1, 'a': 2, 'n': 1}`
6	a	**yes** → 2 + 1	`{'b': 1, 'a': 3, 'n': 2}`

🔑 Lists vs dictionaries

List = find by **position** (`x[0]`), for a sequence of similar things. Dict = find by **key** (`d["name"]`), for labelled data, lookups and counting. Phone numbers stay **text** (`"9500262191"`): they're labels, not quantities.

9 · Parsing input ⭐

↑ contents

Input always arrives as **text**. Parsing = turning it into numbers and containers.

Input	Code	Result
`3`	`n = int(input())`	`3`
`2 3 6`	`input().split()`	`['2', '3', '6']` (still text!)
`2 3 6`	`list(map(int, input().split()))`	`[2, 3, 6]`
`Harsh 25 26.5`	`name, *rest = input().split()`	`"Harsh"`, `['25', '26.5']`
`insert 0 5`	`parts = input().split()`	`parts[0]` = command, `int(parts[1])`

Read inside → out: `list(map(int, input().split()))`

`input()` → `"2 3 6"` · `.split()` → `["2", "3", "6"]` · `map(int, …)` → a lazy "recipe" · `list(…)` → `[2, 3, 6]`

```
student_marks = {}
for _ in range(n):                          # many lines → a dictionary
    name, *line = input().split()
    student_marks[name] = list(map(float, line))
query_name = input()
marks = student_marks[query_name]      # use the ASKED name, after reading it
print(f"{sum(marks) / len(marks):.2f}")
```

❌ What I did

```
arr = map(int, input().split())
arr.sort()           # 'map' object has no attribute 'sort'
parts = input.split()  # missing () → AttributeError
lst = ""             # a string, not a list
```

✅ Right

```
arr = list(map(int, input().split()))
arr.sort()
parts = input().split()
lst = []
```

🔍 **"What am I holding?"**

When confused: `print(x)` and `print(type(x))`. Read nested data one step at a time: `d["Harsh"]` → the list → `d["Harsh"][1]` → one number.

10 · Pattern library

↑ contents

ACCUMULATOR

```
total = 0
for num in nums:
    total += num
```

Running Sum, sum 1–100, vowel counter

BUILD A LIST / FILTER

```
even = []
for num in nums:
    if num % 2 == 0:
        even.append(num)
```

Even filter, Concatenation of Array

COUNTING

```
counts = {}
for x in items:
    counts[x] = counts.get(x, 0) + 1
```

Letter counter, word frequency

BEST SO FAR

```
best = None
for x in items:
    if best is None or x > best:
        best = x
```

Most common letter, second-highest

TWO POINTERS

```
left, right = 0, len(s) - 1
while left < right:
    s[left], s[right] = s[right], s[left]
    left += 1
    right -= 1
```

Reverse String, palindrome

EARLY EXIT · GUARD CLAUSE

```
for x in sorted_scores:
    if x > lowest:
        second = x
        break          # first match only
# guard: if not nums: return 0
```

Runner-Up, Nested Lists, average([])

FLAG

```
ok = True
for x in items:
    if x > top:
        ok = False
        break
print("Yes" if ok else "No")
```

Piling Up! (one answer per test case)

FIND THE BOTTLENECK

```
best = 0
for ch in n:
    if int(ch) > best:
        best = int(ch)
# or: int(max(n))
```

1689: the biggest digit forces the count

Principle	Meaning
USE A BUILT-IN	before writing a loop, ask "does Python have this?" (`[::-1]` , `.replace` , `.insert` , `sum`)
DRY	don't repeat yourself: compute once (`w = word.lower()`) and reuse it
Good starting value	`total = 0` removes "first round" special cases
Correct → readable → efficient	efficient = less work as the input grows (O(n)), not milliseconds. Gauss: `n * (n + 1) // 2`

11 · Error decoder

↑ contents

Read the **last line** of a traceback first (the error), then the line number above it.

Error message	What it really means	Usual fix
`SyntaxError`	Python can't read the line	missing comma, bracket or colon
`IndexError: list index out of range`	position doesn't exist	item used as a position? empty list? use `append`
`IndexError: list assignment index out of range`	`x[i] = …` on a slot that doesn't exist	`x.append(…)`
`KeyError: 'price'`	key not in the dictionary	`.get(key, default)` or `if key in d`
`TypeError: string indices must be integers`	used a letter as a position	use the letter directly
`TypeError: can only concatenate str…`	text + number	`int()` or `str()`
`TypeError: 'list' object cannot be interpreted as an integer`	`range(nums)`	`range(len(nums))`
`TypeError: object of type 'type' has no len()`	`len(list)`: used Python's type name	use your variable (`nums`)
`TypeError: 'builtin_function_or_method' object is not subscriptable`	`[ ]` on a function or method	call methods with `( )`
`AttributeError: 'map' object has no attribute 'sort'`	`map()` isn't a list	`list(map(…))`
`AttributeError: 'list' object has no attribute 'max'`	`max` is a function	`max(x)`
`AttributeError: … has no attribute 'split'` on input	`input.split()`	`input().split()`
`NameError: name 'x' is not defined`	variable doesn't exist here (typo, or made inside a function)	check spelling / `return` it
`ZeroDivisionError`	divided by 0 (e.g. `len([])`)	guard clause
`EOFError`	`input()` had nothing to read	provide input / custom test input

Time Limit Exceeded	code never finishes	`while` condition never becomes False
Wrong Answer, 1 test passes	passed by luck	ask "what's different about the failing input?"

12 · LeetCode & HackerRank format

↑ contents

🟠 LeetCode

```
class Solution:
    def runningSum(self, nums: list[int]) -> list[int]:
        ...                 # just write logic here
        return output       # RETURN, don't print
```

- Ignore `class`, `self`, `-> int` for now
- **"In place"** → change the given list, return nothing
- "Beats X%" on tiny problems = timer noise. Resubmitting changes it
- Streak = **profile "Max streak"** (any accepted problem). The 🔥 top-bar counter = Daily Question only

🔥 HackerRank

```
if __name__ == '__main__':
    n = int(input())    # keep their lines
    ...                 # write BELOW them
    print(answer)       # PRINT (no return here)
```

- Output must match **exactly** (case, spaces)
- Scroll the output box: lines can be hidden
- Run Code first, then Submit Code
- Language: **Python 3**

13 · Git daily commands

↑ contents

Goal	VS Code	Terminal
stage one file	hover the file row → its own **+**	`git add file.py`
commit	message box (**Ctrl+A, Delete** first!) → ✓ **Commit**	`git commit -m "msg"`
push	🔄 **0↓ N↑** in the status bar	`git push`
check status	Source Control panel	`git status`
stage part of a file	select lines → **Ctrl+Shift+P** → **Git: Stage Selected Ranges**	(new files: ask Claude)

🔑 Commit ≠ push

Commit saves to the laptop. **Push/Sync** sends to GitHub. Only pushed commits make the square green, and the graph can take a while to update.

🟩 Good commits

One logical change per commit, with a clear message: `Day 05: word counter with split()`. Never fake or back-date commits.

14 · Problems solved

↑ contents

🟠 LeetCode

Date	Problem	Pattern / lesson
2026-10-17	2235. Add Two Integers	✅ Day 1 🔥 (profile: active days 1, max streak 1)
2026-10-18	1929. Concatenation of Array	✅ Day 2 🔥 (after fixing items vs positions + append)
2026-10-19	1480. Running Sum of 1d Array	✅ Day 3 🔥 (ACCUMULATOR; fixed `output[value - 1]`)
2026-10-20	1108. Defanging an IP Address	✅ Day 4 🔥 (own loop, first file in `leetcode/`)
2026-10-21	344. Reverse String	✅ Day 5 🔥 (two pointers; fixed an infinite loop + double reversal)
2026-10-22	2149. Rearrange Array Elements by Sign	✅ Day 6 🔥 **first Medium!** (two FILTER lists + take turns; fixed IndexError from mixing big/small list positions)
2026-10-23	2161. Partition Array According to Given Pivot	✅ Day 7 🔥 Medium (3 FILTER lists + `+`; fixed `pivot = []` overwriting the parameter and `nums` vs `nums[i]`)
2026-10-24	1672. Richest Customer Wealth	✅ Day 8 🔥 Easy (nested list + BEST SO FAR; fixed `range(list)` and a missing `:`; 0 ms)
2026-10-25	412. Fizz Buzz	✅ Day 9 🔥 Easy (exclusive conditions; fixed `range(1:n+1)`, `elif elif`, `str(i)`; 0 ms)
2026-10-26	1689. Partitioning Into Minimum Number Of Deci-Binary Numbers	✅ Day 10 🔥 Medium (own greedy on paper → max digit; BEST SO FAR; 0 ms)
2026-10-27	42. Trapping Rain Water	✅ Day 11 🔥 🔴 **first Hard!** (picture → fill-in-the-blank steps; left/right BEST SO FAR rows; assembled + pasted, retype later; 13 ms)

🔥 HackerRank

Date	Problem	Pattern / lesson
2026-10-13	Python If-Else	✅
2026-10-13	Arithmetic Operators	✅
2026-10-13	Python: Division	✅ ⭐ 1st star
2026-10-13	Loops	✅ (after an off-by-one fix)
2026-10-14	Print Function	✅ ⭐⭐ 2nd star
2026-10-15	Find the Runner-Up Score!	✅ (after fixing `map` → `list(map( ... ))` and adding `break`)
2026-10-16	Write a function (leap year)	✅ solved on my phone 📱 (90/110 pts, 20 to ⭐⭐⭐)
2026-10-17	Finding the percentage	✅ (after fixing a leftover loop variable: `name` → `query_name`)
2026-10-18	Nested Lists	✅ using the STEP-comments method (steps 3–5 written myself)
2026-10-21	Lists	✅ (function per command + if/elif dispatch; built-in .insert/.remove + int())
2026-10-22	Word Order	✅ **first Medium** (COUNTING pattern; fixed counting letters instead of words)
2026-10-23	Merge the Tools!	✅ Medium (own window logic first; walkthrough for `ch not in result` + reset per piece)
2026-10-26	Piling Up!	✅ Medium ⭐⭐⭐⭐ **4th star** (own pop-from-the-ends idea; fixed `pop(value)`, added top check + `ok` flag after a walkthrough)

Generated from `cheatsheet/cheatsheet.src.html` · updated 27 Oct 2026 · ↑ back to cover